PYTHON ACTIVITY-BASED LEARNING

10 Beginner-Friendly Case Studies (Questions 21–30)

Prepared according to the assignment circular: each case includes a problem statement, objective, assumptions, algorithm, flowchart, pseudocode, Python program, test case, and conclusion.

Flowchart symbol guide

Oval	Start / Stop
Parallelogram	Input / Output
Rectangle	Processing / calculation
Diamond	Decision (Yes / No)

Important: Fine amounts, chemical limits, tariffs, and validation rules below are invented sample rules for learning Python. Replace them only with rules supplied by your teacher.

Case Study 21: Traffic Violation Penalty Calculator

1. Problem statement: Calculate a total fine for several violations using simple example rules and flag a repeat offence.

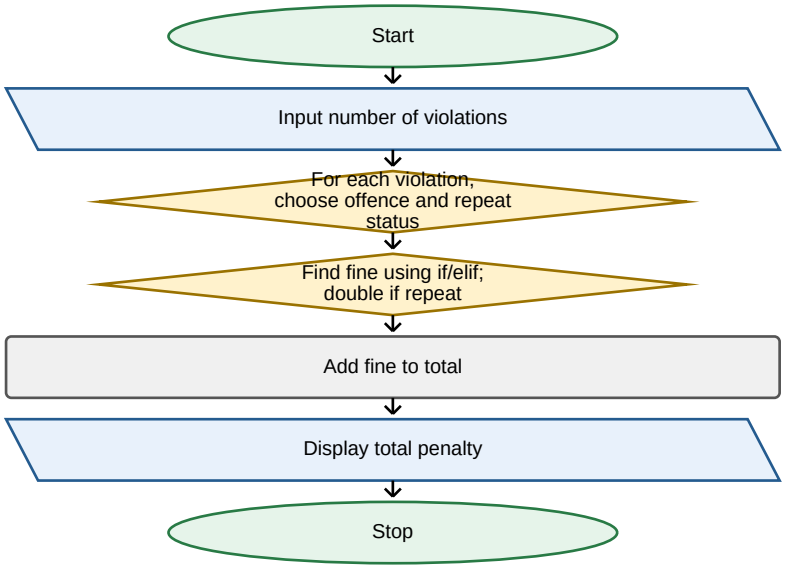
2. Objective: Use lists, loops, if/elif decisions, and arithmetic to total penalties.

3. Assumptions: Example rules only: speeding = ■1,000; no helmet = ■500; signal jump = ■1,500. Repeat offence doubles that offence's fine. These are classroom sample values, not legal advice.

4. Algorithm

1. Start
2. Input number of violations
3. For each violation, choose offence and repeat status
4. Find fine using if/elif; double if repeat
5. Add fine to total
6. Display total penalty
7. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
total = 0\nRepeat for each violation:\n    Read offence and repeat status\n    Select base fine using if/elif\n    If repea
```

7. Python source code

```
rules = {"speeding": 1000, "helmet": 500, "signal": 1500}
total = 0
count = int(input("How many violations? "))

for i in range(count):
    offence = input("Enter speeding, helmet, or signal: ").lower()
    repeat = input("Repeat offence? (yes/no): ").lower()
    if offence in rules:
        fine = rules[offence]
        if repeat == "yes":
            fine = fine * 2
        total += fine
    else:
        print("Unknown offence; skipped")

print("Total penalty: Rs.", total)
```

8. Test case and expected result

Input: 2 violations: helmet, no; signal, yes. Expected total: ■500 + ■3,000 = ■3,500.

9. Output

```
Total penalty: Rs. 3500
```

10. Conclusion

The program totals multiple sample fines and applies a repeat-offence rule.

Case Study 22: Laboratory Chemical Inventory Validation

1. Problem statement: Check hypothetical inventory records against maximum allowed quantities and storage categories.

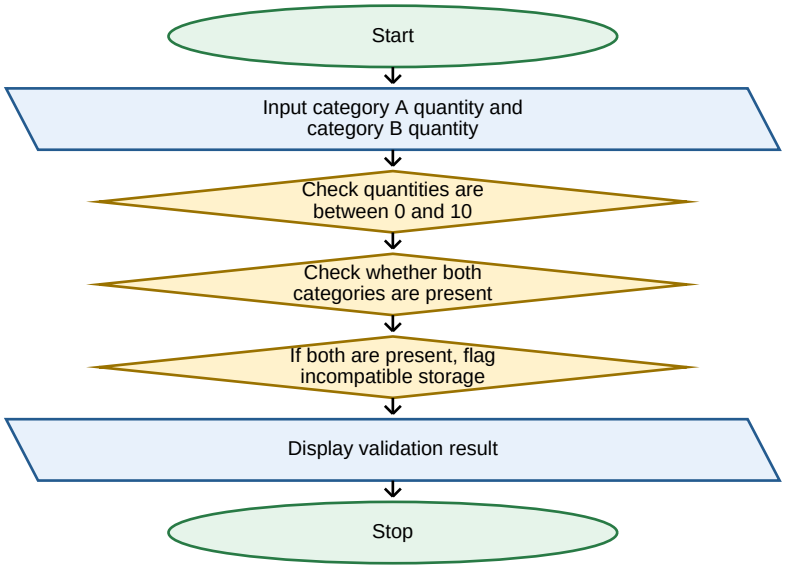
2. Objective: Validate quantity and storage category using conditions.

3. Assumptions: This is a simplified classroom example. Categories are A and B; do not store A and B together. Maximum quantity is 10 units per category.

4. Algorithm

1. Start
2. Input category A quantity and category B quantity
3. Check quantities are between 0 and 10
4. Check whether both categories are present
5. If both are present, flag incompatible storage
6. Display validation result
7. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
Read quantity_A, quantity_B\nIf either quantity is outside 0..10: print invalid quantity\nElse if both quantities are great
```

7. Python source code

```
a = int(input("Quantity in category A: "))
b = int(input("Quantity in category B: "))

if a < 0 or b < 0 or a > 10 or b > 10:
    print("Invalid quantity")
elif a > 0 and b > 0:
    print("Flag: store categories A and B separately")
else:
    print("Inventory passes the sample checks")
```

8. Test case and expected result

Input A=4, B=2. Expected: Flag: store categories A and B separately.

9. Output

```
Flag: store categories A and B separately
```

10. Conclusion

The program checks sample quantity limits and a supplied compatibility rule. Real laboratories must follow official safety guidance.

Case Study 23: Student Timetable Conflict Detection

1. Problem statement: Detect overlapping course periods and check whether a room is unavailable.

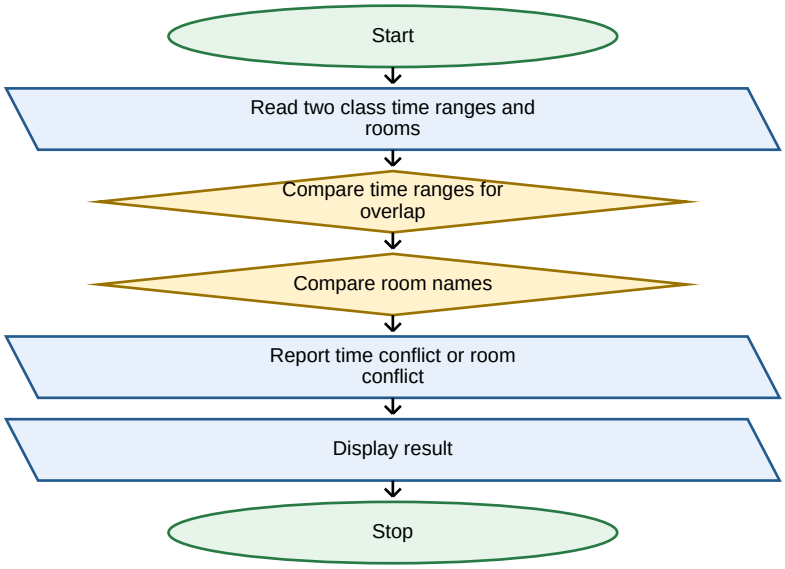
2. Objective: Use tuples, lists, and comparisons to identify schedule conflicts.

3. Assumptions: Each class is a tuple (course, start_hour, end_hour, room). Times are whole hours; a class overlaps another if start1 < end2 and start2 < end1.

4. Algorithm

1. Start
2. Read two class time ranges and rooms
3. Compare time ranges for overlap
4. Compare room names
5. Report time conflict or room conflict
6. Display result
7. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
Read course 1 start/end/room\nRead course 2 start/end/room\nIf start1 < end2 and start2 < end1: report time overlap\nIf roo
```

7. Python source code

```
c1 = ("Python", 9, 11, "R1")
c2 = ("Math", 10, 12, "R1")

overlap = c1[1] < c2[2] and c2[1] < c1[2]
same_room = c1[3] == c2[3]

if overlap:
    print("Time conflict detected")
    if same_room:
        print("Room conflict:", c1[3])
else:
    print("No time conflict")
```

8. Test case and expected result

Class 1: 9–11 in R1; class 2: 10–12 in R1. Expected: time and room conflict.

9. Output

```
Time conflict detected\nRoom conflict: R1
```

10. Conclusion

The program demonstrates how tuples and comparison operators can detect simple timetable clashes.

Case Study 24: Courier Charge Optimization

1. Problem statement: Calculate courier prices for different services and choose the cheapest eligible service.

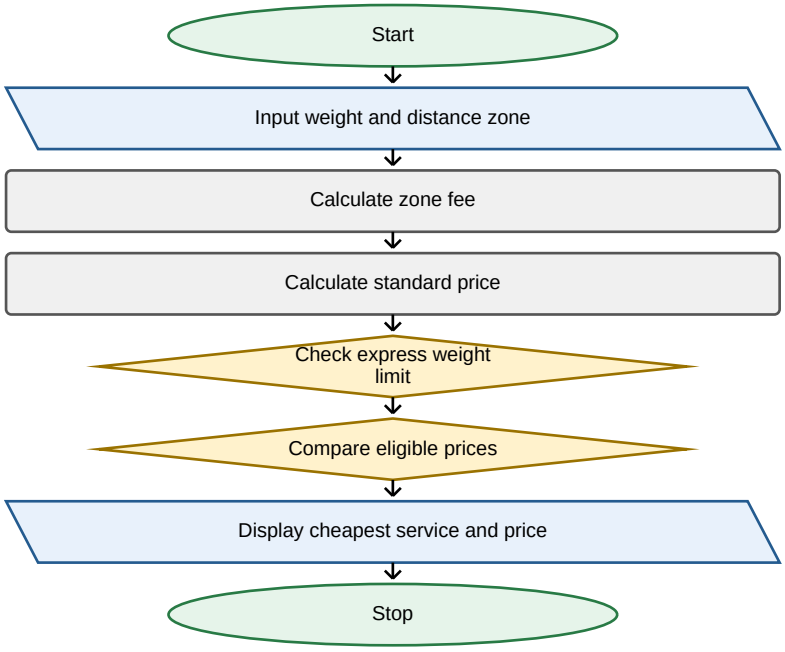
2. Objective: Use conditions and arithmetic to compare service options.

3. Assumptions: Sample rates: Standard = ■40 + ■10 per kg; Express = ■70 + ■15 per kg. Express is unavailable above 5 kg. Distance zones are represented by an extra fee: local ■0, regional ■30, national ■60.

4. Algorithm

1. Start
2. Input weight and distance zone
3. Calculate zone fee
4. Calculate standard price
5. Check express weight limit
6. Compare eligible prices
7. Display cheapest service and price
8. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
Read weight and zone\nFind zone fee\nstandard = 40 + 10*weight + zone_fee\nIf weight <= 5: express = 70 + 15*weight + zone_
```

7. Python source code

```
weight = float(input("Parcel weight in kg: "))
zone = input("Zone (local/regional/national): ").lower()

fees = {"local": 0, "regional": 30, "national": 60}
if weight <= 0 or zone not in fees:
    print("Invalid weight or zone")
else:
    extra = fees[zone]
    standard = 40 + 10 * weight + extra
    print("Standard: Rs.", standard)

    if weight <= 5:
        express = 70 + 15 * weight + extra
        print("Express: Rs.", express)
        if standard <= express:
            print("Cheapest eligible option: Standard")
        else:
            print("Cheapest eligible option: Express")
    else:
        print("Express unavailable; choose Standard")
```

8. Test case and expected result

Input 2 kg, local. Standard ■60; Express ■100. Expected: Standard is cheapest.

9. Output

```
Standard: Rs. 60.0\nExpress: Rs. 100.0\nCheapest eligible option: Standard
```

10. Conclusion

The program compares two sample courier prices and respects a simple service limit.

Case Study 25: Progressive Water Tariff Analysis

1. Problem statement: Calculate household water bills using consumption slabs and a fixed fee.

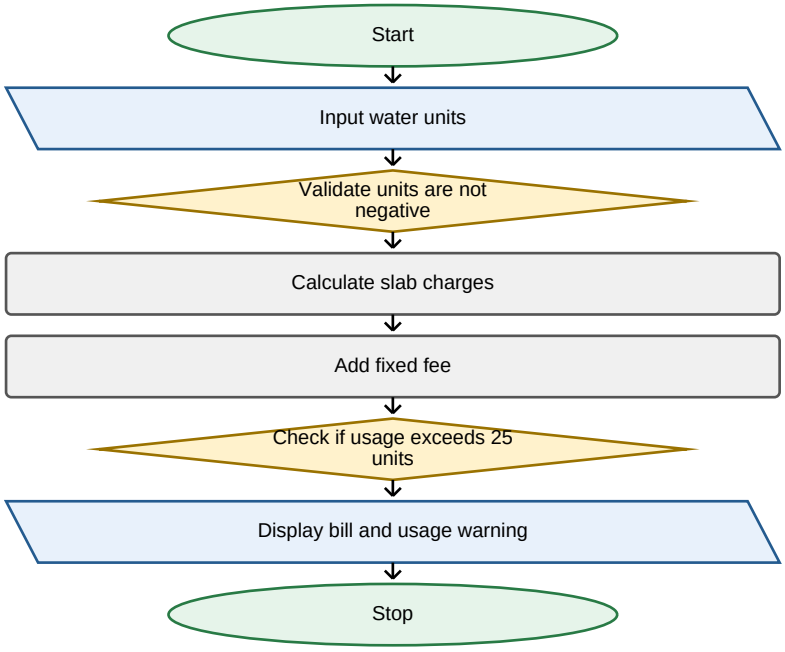
2. Objective: Apply slab-based billing and identify high consumption.

3. Assumptions: First 10 units cost ■2/unit; next 10 units cost ■3/unit; above 20 units cost ■5/unit. Fixed fee ■20. Flag usage above 25 units.

4. Algorithm

1. Start
2. Input water units
3. Validate units are not negative
4. Calculate slab charges
5. Add fixed fee
6. Check if usage exceeds 25 units
7. Display bill and usage warning
8. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
Read units\nIf units < 0: show invalid input\nElse calculate slab cost\nAdd fixed fee 20\nIf units > 25: print high-usage w
```

7. Python source code

```
units = float(input("Water units used: "))

if units < 0:
    print("Units cannot be negative")
else:
    if units <= 10:
        charge = units * 2
    elif units <= 20:
        charge = 10 * 2 + (units - 10) * 3
    else:
        charge = 10 * 2 + 10 * 3 + (units - 20) * 5

    bill = charge + 20
    print("Water bill: Rs.", bill)
    if units > 25:
        print("Warning: high water usage")
```

8. Test case and expected result

Input 25 units. Charge = ■20 + ■30 + ■25 + fixed ■20 = ■95. No warning because usage is not above 25.

9. Output

```
Water bill: Rs. 95.0
```

10. Conclusion

The program calculates a sample progressive tariff and flags usage above the chosen threshold.

Case Study 26: Nested Loop Examination Seating Generator

1. Problem statement: Create a seating grid and mark reserved seats.

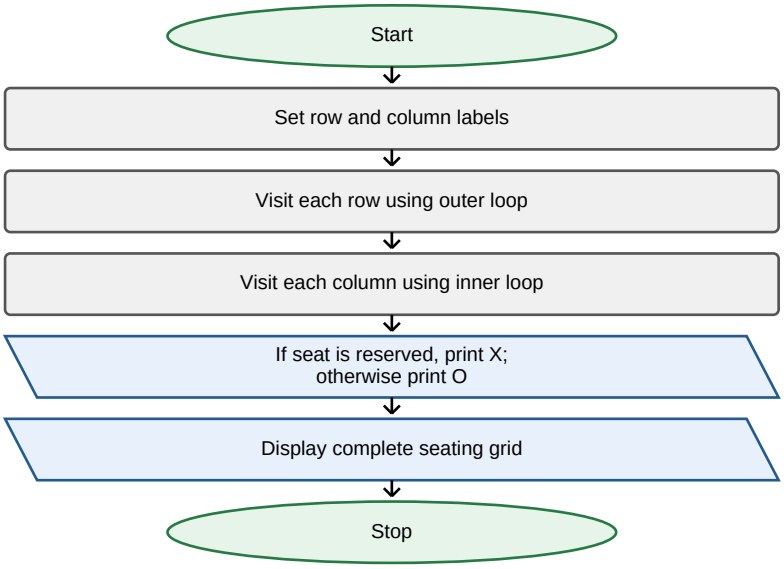
2. Objective: Practise nested loops, row/column labels, and membership checks.

3. Assumptions: Use 3 rows and 4 columns. Seat B2 is reserved. Rows are A, B, C and columns are 1–4.

4. Algorithm

1. Start
2. Set row and column labels
3. Visit each row using outer loop
4. Visit each column using inner loop
5. If seat is reserved, print X; otherwise print O
6. Display complete seating grid
7. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
rows = A, B, C; columns = 1..4\nFor each row:\n    For each column:\n        If seat is reserved: print X\n        Else: pr
```

7. Python source code

```
rows = ["A", "B", "C"]
columns = [1, 2, 3, 4]
reserved = ["B2"]

for row in rows:
    for col in columns:
        seat = row + str(col)
        if seat in reserved:
            print("X", end=" ")
        else:
            print("O", end=" ")
    print()
```

8. Test case and expected result

Expected grid: O O O O / O X O O / O O O O. X means reserved; O means available.

9. Output

```
O O O O\nO X O O\nO O O O
```

10. Conclusion

Nested loops can generate a seating layout and mark seats that cannot be assigned.

Case Study 27: Advanced Numeric Pattern Generator

1. Problem statement: Print a hollow diamond-like pattern using stars for a user-provided odd size.

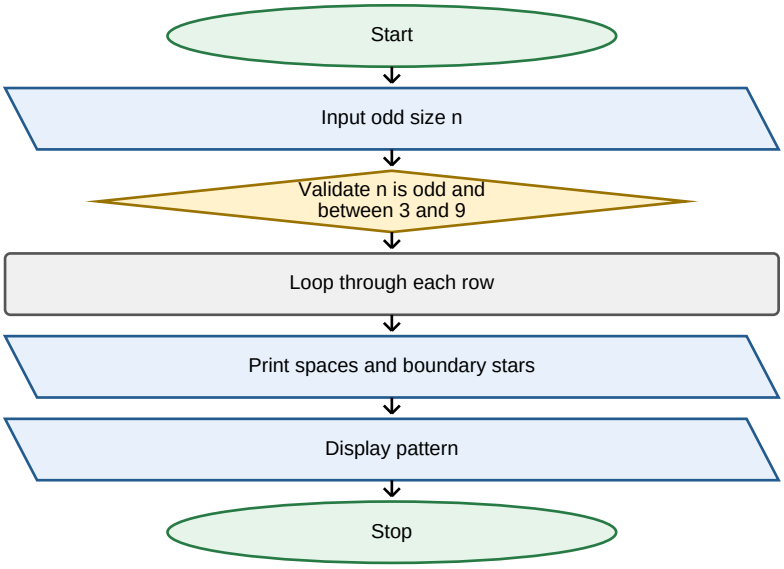
2. Objective: Use loops and conditions to create aligned output.

3. Assumptions: Size must be an odd integer from 3 to nine. This beginner version prints a hollow diamond pattern.

4. Algorithm

1. Start
2. Input odd size n
3. Validate n is odd and between 3 and 9
4. Loop through each row
5. Print spaces and boundary stars
6. Display pattern
7. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
Read n\nIf n < 3 or n > 9 or n is even: show invalid size\nElse for each row i from 0 to n-1:\n    calculate distance from
```

7. Python source code

```
n = int(input("Enter an odd size from 3 to 9: "))

if n < 3 or n > 9 or n % 2 == 0:
    print("Invalid size")
else:
    mid = n // 2
    for i in range(n):
        distance = abs(mid - i)
        outer = distance
        inner = n - 2 * distance - 2
        print(" " * outer, end="")
        if inner < 0:
            print("*")
        else:
            print("*" + " " * inner + "*")
```

8. Test case and expected result

Input 5. Expected output is a symmetric outline pattern; even numbers such as 4 are rejected.

9. Output

```
A symmetric hollow pattern is printed for size 5.
```

10. Conclusion

The program uses loops, arithmetic, and validation to build a simple pattern.

Case Study 28: Repeated Transaction Detection

1. Problem statement: Find duplicate transaction IDs and flag transactions above a limit.

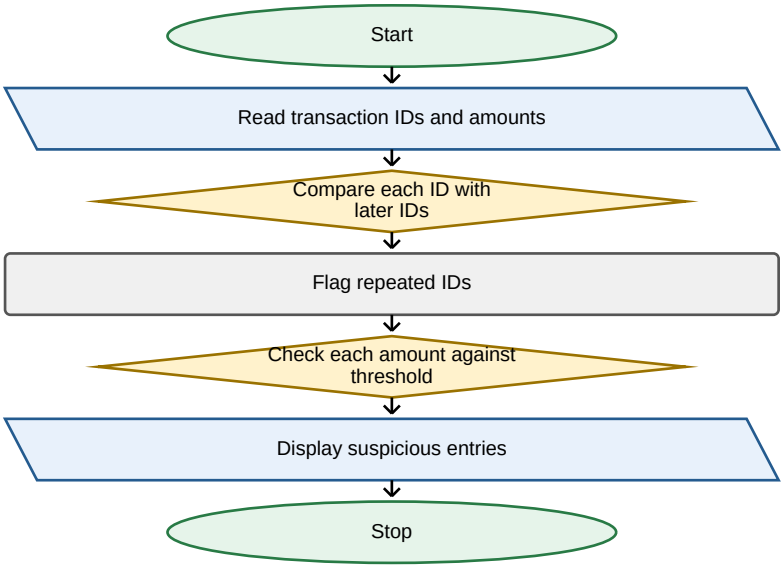
2. Objective: Search lists and report duplicate IDs or large amounts.

3. Assumptions: A transaction is high-value if amount is above ■10,000. A repeated ID is considered a possible duplicate.

4. Algorithm

1. Start
2. Read transaction IDs and amounts
3. Compare each ID with later IDs
4. Flag repeated IDs
5. Check each amount against threshold
6. Display suspicious entries
7. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
ids = list of IDs; amounts = matching list\nFor each index i:\n    If ID appears more than once: report duplicate\n    If a
```

7. Python source code

```
ids = ["T101", "T102", "T101", "T103"]
amounts = [500, 12000, 500, 15000]

for i in range(len(ids)):
    if ids.count(ids[i]) > 1:
        print("Possible duplicate:", ids[i])
    if amounts[i] > 10000:
        print("High-value transaction:", ids[i], amounts[i])
```

8. Test case and expected result

Expected: T101 flagged as duplicate; T102 and T103 flagged as high-value transactions.

9. Output

```
Possible duplicate: T101\nHigh-value transaction: T102 12000\nPossible duplicate: T101\nHigh-value transaction: T103 15000
```

10. Conclusion

The program identifies simple warning signs. A flag is not proof of fraud.

Case Study 29: Password Policy Validation Engine

1. Problem statement: Check a sample password against minimum length, digits, uppercase letters, and repeated characters.

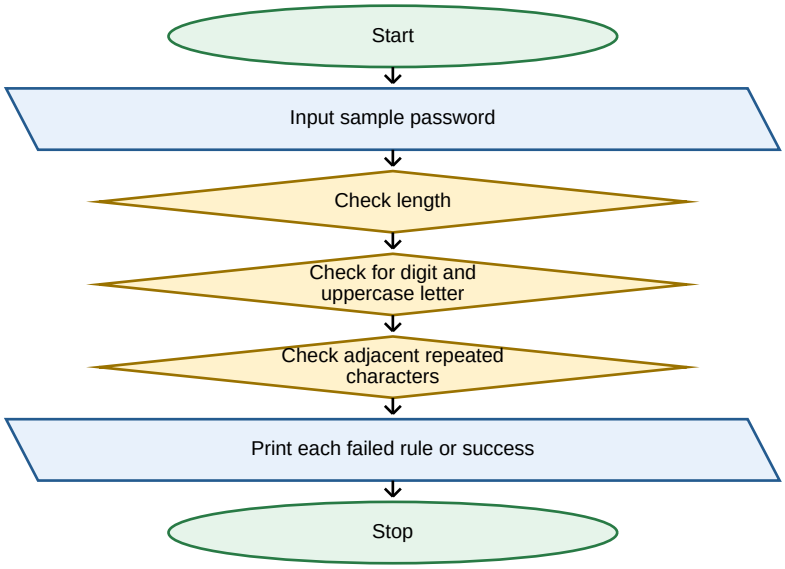
2. Objective: Use string methods and conditions to explain failed checks.

3. Assumptions: For this exercise only, require at least 8 characters, one digit, one uppercase letter, and no identical adjacent characters.

4. Algorithm

1. Start
2. Input sample password
3. Check length
4. Check for digit and uppercase letter
5. Check adjacent repeated characters
6. Print each failed rule or success
7. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
Read password\nIf length < 8: report length failure\nIf no digit: report digit failure\nIf no uppercase: report uppercase f
```

7. Python source code

```
password = input("Enter a sample password: ")
errors = []

if len(password) < 8:
    errors.append("Use at least 8 characters")
if not any(ch.isdigit() for ch in password):
    errors.append("Add a digit")
if not any(ch.isupper() for ch in password):
    errors.append("Add an uppercase letter")

for i in range(len(password) - 1):
    if password[i] == password[i + 1]:
        errors.append("Avoid identical adjacent characters")
        break

if errors:
    for error in errors:
        print(error)
else:
    print("Sample policy passed")
```

8. Test case and expected result

Input: abcdefgh. Expected: asks for a digit and uppercase letter; length passes.

9. Output

```
Add a digit\nAdd an uppercase letter
```

10. Conclusion

The program practises string validation. It is a classroom example, not a complete security checker.

Case Study 30: Student Email and Identifier Validation

1. Problem statement: Check a student email-like string and registration ID against basic formatting rules.

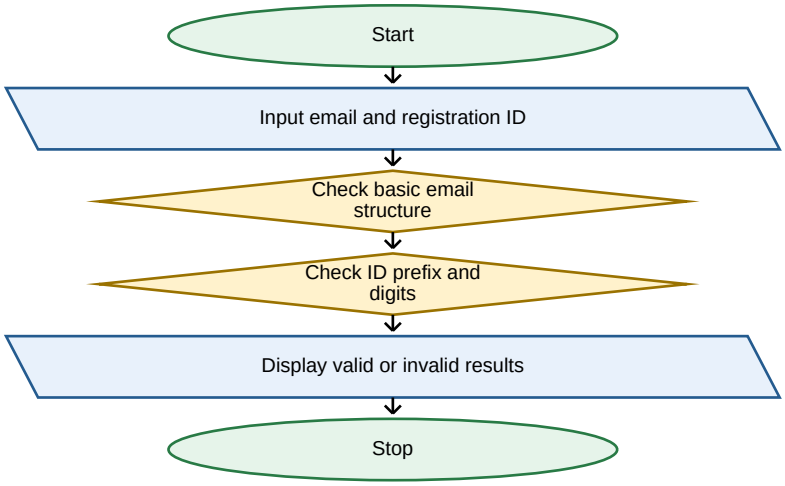
2. Objective: Use string methods and if/else statements to validate simple formats.

3. Assumptions: Email example must contain exactly one @ and a dot after @. Registration ID must begin with 'STU' and then contain digits, e.g. STU1234.

4. Algorithm

1. Start
2. Input email and registration ID
3. Check basic email structure
4. Check ID prefix and digits
5. Display valid or invalid results
6. Stop

5. Flowchart (standard shapes)



Oval = Start/Stop; parallelogram = Input/Output; rectangle = process; diamond = decision.

6. Pseudocode

```
Read email and ID\nEmail valid if one @, dot after @, and no spaces\nID valid if starts with STU and remaining characters a
```

7. Python source code

```
email = input("Enter student email: ")
student_id = input("Enter ID (example STU1234): ")

at_count = email.count("@")
if at_count == 1 and "." in email.split("@")[1] and " " not in email:
    print("Email format: valid")
else:
    print("Email format: invalid")

if student_id.startswith("STU") and student_id[3:].isdigit():
    print("Student ID: valid")
else:
    print("Student ID: invalid")
```

8. Test case and expected result

Input email student@example.com and ID STU1234. Expected: both formats valid under the sample rules.

9. Output

```
Email format: valid\nStudent ID: valid
```

10. Conclusion

The program checks simple formatting rules; it does not confirm that an email address actually exists.